



Ch 4: Constraint Satisfaction Problems

Artificial Intelligence

Dr. Udaya R. Dhungana
Assistant Professor
Pokhara University, Nepal

Former Guest Faculty, Darmstadt University of Applied Sciences, Germany



Overview

Ch 4: Constraint Satisfaction
Problems (5 hrs)

1. Defining CSPs
2. Constraint Propagation
3. Inference in CSPs
4. Backtracking Search for CSPs

Problem Solving by Searching

- Standard Search Strategies:
 - uninformed or informed or local or adversarial search algorithm
 - Solve problems by searching a state space
 - the state space is represented by search tree (or graph)- nodes represent the states and edges represent the actions.
 - Domain specific heuristic functions are used to estimate the cost of reaching the goal state from a given state
 - Each *state* is considered as the *atomic* or indivisible- a **black box** with no internal structure
- In this chapter we break the **black box** (each state) into a set of *variables*, a set of *domain* values, a set of *constrains*.



Constraints Satisfaction Problems

- each *state* are *not atomic*
- *each state* contains
 - a set of *variables*
 - each variable has a value from a set of *domain* values
 - Each variable and its value satisfy a set of *constraints*.
- A problem is solved when each variable is assigned a value that satisfies all the constraints on the variable.
- *Idea of CSPs*: to eliminate large portions of the search space all at once by identifying variable/value combinations that violate the constraints.



Constraints Satisfaction Problems

- Standard search problem:
 - **state** is a "black box" - any data structure that supports successor function, heuristic function, and goal test
 - Problems can be solved by searching in a space of states
- CSP:
 - **state** is defined by **variables** X_i with **values** V_i from **domain** D_i
 - **goal test** is a set of **constraints** specifying allowable combinations of values for subsets of variables



Constraints Satisfaction Problems

- Many real-world problems—from scheduling flights to solving puzzles—can be naturally modeled as **Constraint Satisfaction Problems (CSPs)**.
- Unlike general search problems that use arbitrary representations, CSPs express problems in a **structured and uniform way**, leading to more **efficient solving techniques**.



Defining CSPs

- A constraint satisfaction problem is defined by 3 components , X , D , C :
 - X is a set of variables, $\{X_1, \dots, X_n\}$.
 - D is a set of domain, $\{D_1, \dots, D_n\}$, one for each variable.
 - C is a set of constraints that specify allowable combinations of values.
- A solution is an assignment of values to all variables such that **every constraint is satisfied**.



Defining CSPs

- A domain D_i consists of a set of allowable values, $\{v_1, \dots, v_k\}$ for a variable X_i .
 - E.g. a boolean variable has the domain $\{true, false\}$.
 - So different variables can have different domains of different sizes.
- Each constraint C_j consists a pair $\langle scope, rel \rangle$ where
 - *scope* is a tuple of variables that participate in the constraint and
 - *rel* is a relation that defines the values that those variables can take on.



Defining CSPs

If X_1 and X_2 both have the domain $\{1,2,3\}$,

then the constraint saying that X_1 must be greater than X_2 can be written as

$$\langle (X_1, X_2), \{(3,1), (3,2), (2,1)\} \rangle$$

or

$$\langle (X_1, X_2), X_1 > X_2 \rangle$$



Defining CSPs

CSPs deal with assignment of values from the domain to the variables $\{X_i = v_i, X_j = v_j, \dots\}$.

- **Consistent** assignment
 - An assignment that does not violate any constraints is called a **consistent** or **legal** assignment
- **Complete** assignment
 - A **complete assignment** is one in which every variable is assigned a value



Defining CSPs

CSPs deal with assignment of values from the domain to the variables $\{X_i = v_i, X_j = v_j, \dots\}$.

- **Partial** assignment
 - A partial assignment is the one that leaves some variables unassigned.
- **Solution**
 - A solution to a CSP is a consistent, complete assignment.
- **Partial Solution**
 - A **partial solution** is a partial assignment that is consistent

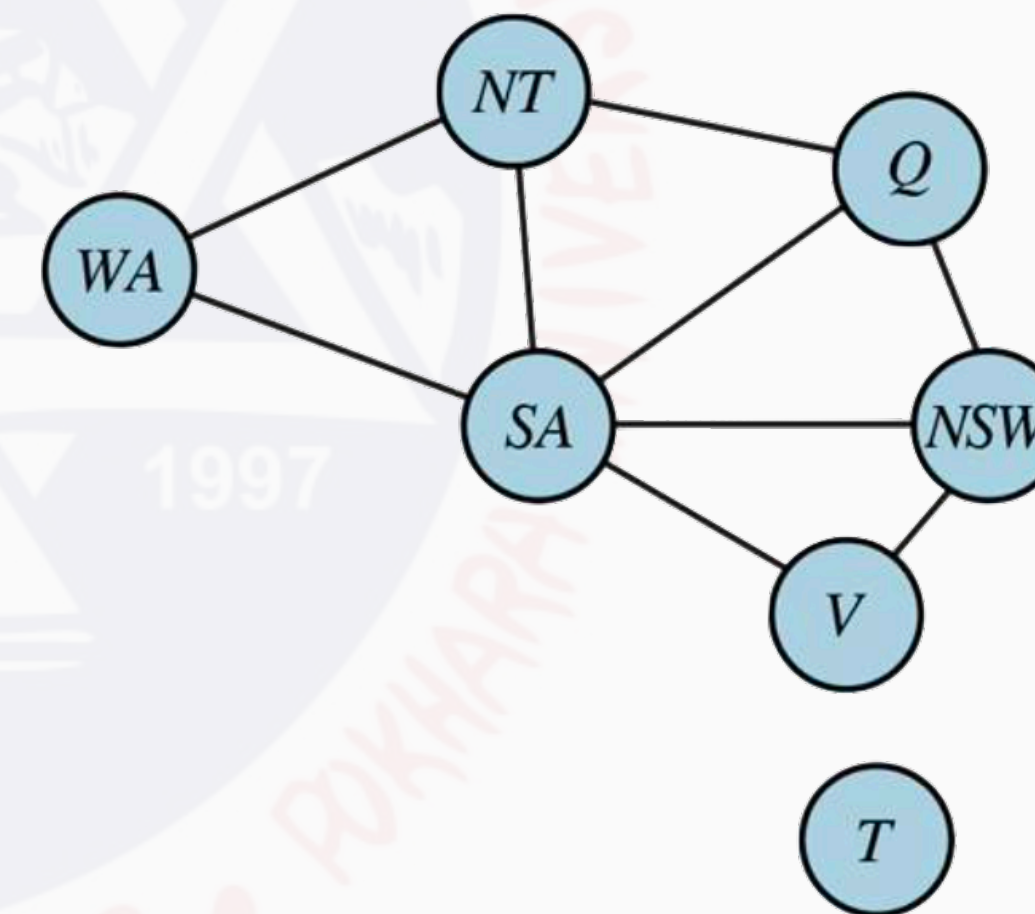


Example Problem: Map Coloring

Suppose we are looking at a map of Australia showing each of its states and territories. We are given the task of coloring each region either red, green, or blue in such a way that no two neighboring regions have the same color.



(a)



(b)

(a) The principal states and territories of Australia. Coloring this map can be viewed as a constraint satisfaction problem (CSP). The goal is to assign colors to each region so that no neighboring regions have the same color. (b) The map-coloring problem represented as a constraint graph.

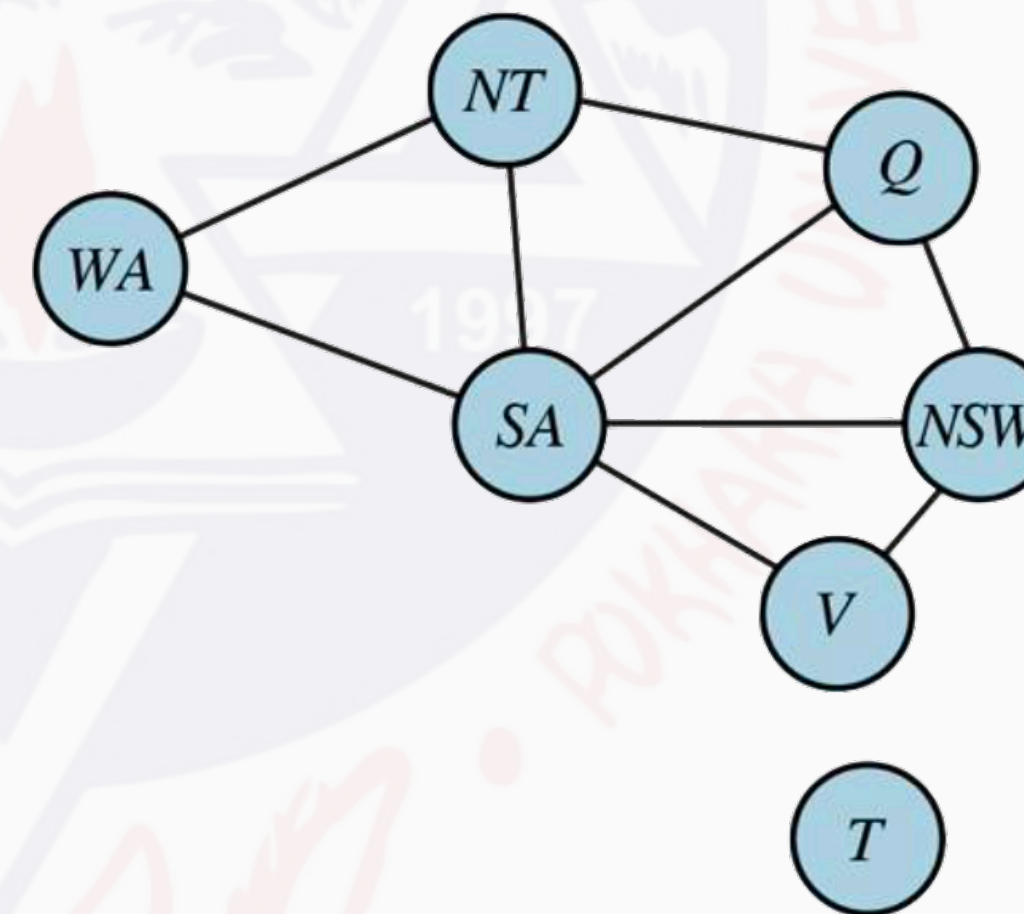


Example Problem: Map Coloring

- **Constraint graph:** nodes are variables, arcs are constraints



(a) Map of Australia



(b) Constraint Graph of (a)

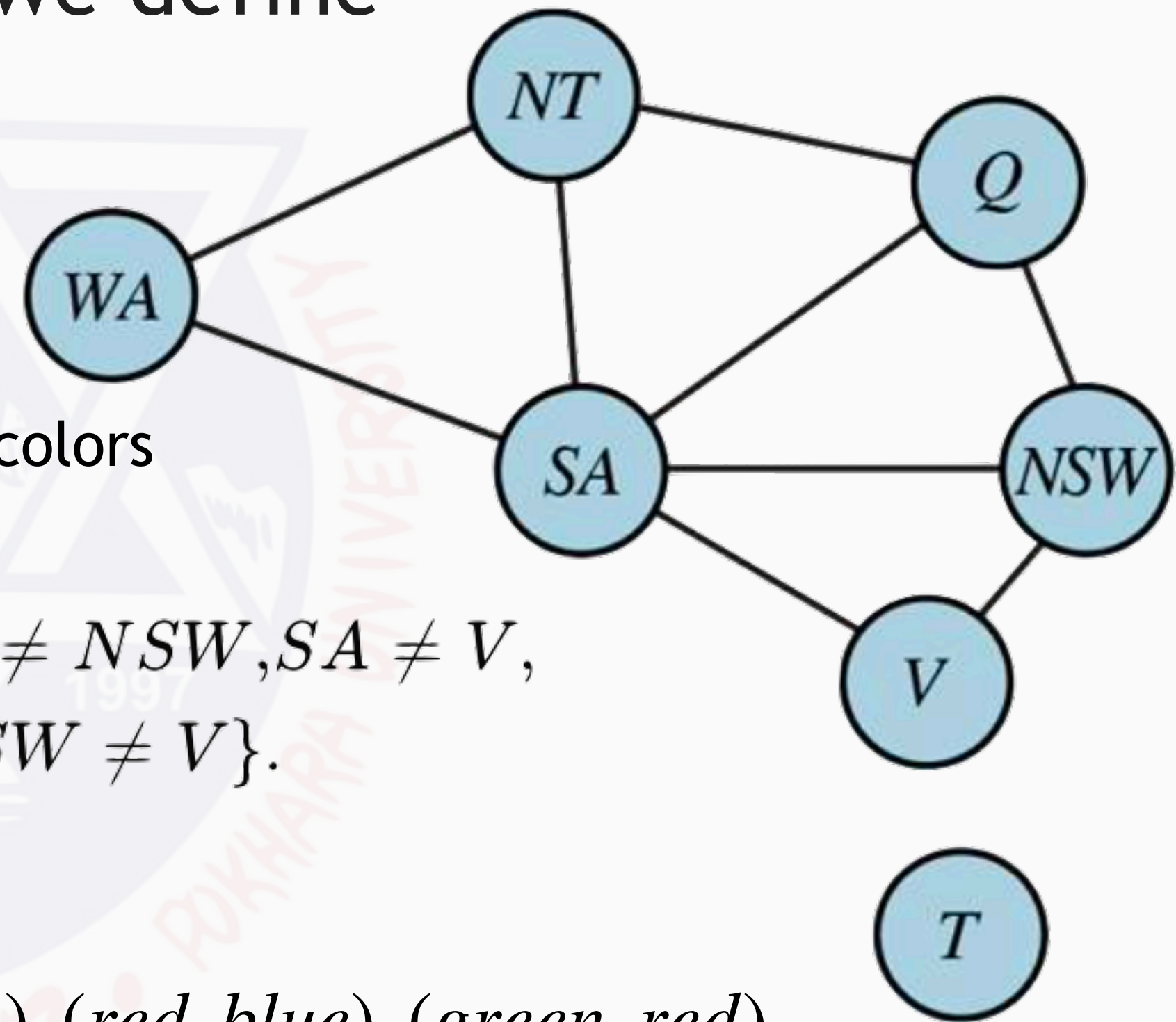


Example Problem: Map Coloring

- To formulate this problem as a CSP, we define

- Variables, $X = \{WA, NT, Q, NSW, V, SA, T\}$
- Domain for each variable $D_i = \{red, green, blue\}$
- Constraints C : adjacent regions must have distinct colors

$$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, \\ WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}.$$



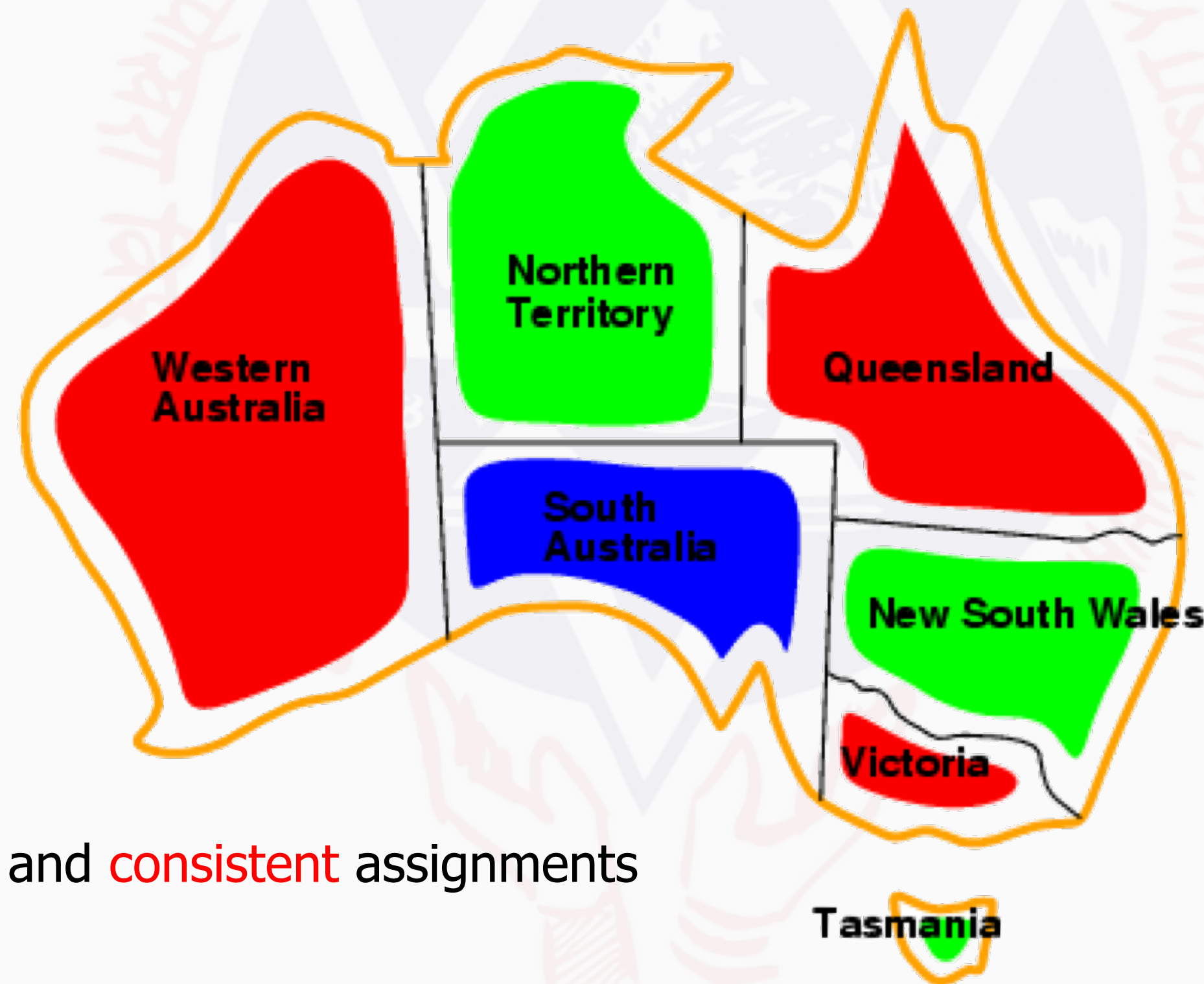
Where, $SA \neq WA \implies \langle (SA, WA), \{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\} \rangle$



Example Problem: Map Coloring

- There are many possible solutions to this problem, such as

$\{WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green\}$



- Solutions are **complete** and **consistent** assignments



Formulating CSP as Search Problem

- **Initial state:**

- the empty assignment $\{\}$ – all variables are unassigned

- **Successor function:**

- a value is assigned to one of the unassigned variables with no conflict
- fail if no legal assignments

- **Goal test:**

- a complete assignment: all variables have a value and none of the constraints is violated.

- **Path cost:**

- a constant cost for each step

- Solution appears at depth n if there are n variables
- Depth-first or local search methods work well
- Path is irrelevant, so can also use complete-state formulation



Why formulate a problem as a CSP?

- *Easy to formulate* a problem as a CSP
- CSP solvers are *fast and efficient*
- A CSP solver can *quickly prune large swathes* of the search space



Why formulate a problem as a CSP?

- Once you assign $\{SA = blue\}$,
 - None of the five neighboring variables can take on the value *blue*. So remaining no of colors = 2, $\{green, red\}$
- A search procedure with the use of no constraints would have to consider $3^5 = 243$ assignments for the 5 neighboring variables
- While with constraints, it would have to consider only $2^5 = 32$ assignments



Types of Constraints

- **Unary constraint**

- restricts the value of a single variable
- Example: Suppose a constraint that SA cannot have green color. That is *SA $\neq$ green*

- **Binary constraint**

- relates two variables with some restriction
- Example: *SA $\neq$ NSW*



Types of Constraints

- **Higher-order constraint**

- restricts the value of more than two variables

- **Example:** the ternary constraint between (X, Y, Z) : $\langle (X, Y, Z), X < Y < Z \rangle$

- **Global constraint**

- A constraint involving an arbitrary number of variables

- **Example:** *Alldiff*

(which says that all of the variables involved in the constraint must have different values)



Real World CSPs

- **Scheduling problems**
 - e.g. Class Scheduling
- **Configuration and DDesign**
 - e.g. Product Configuration (e.g., Cars, Computers), Architectural Design
- **Puzzle and Games**
 - e.g. Map coloring, N-queens, Sudoku
- **Resource Allocation**
 - e.g. machine job assignment



Constraint Propagation: Inference in CSPs

- *In Standard Search:*

- An atomic state-space search algorithm makes progress in only one way: by expanding a node to visit the successors

- *In Constraint Satisfaction Problems:*

- CSPs can generate successors by choosing a new variable assignment
 - For this it can use a specific type of inference called **constraint propagation**
- **Constraint propagation** reduces the number of legal values for a variable
 - which in turn can reduce the legal values for another variable, and so on



Constraint Propagation

- Provides an efficient method in inference process of CSPs
- **Constraint Propagation** is a technique used in **CSPs** to reduce the search space by systematically enforcing constraints during the search process
- It works by iteratively enforcing constraints to eliminate values from the domains of variables that cannot be part of any valid solution.
- eliminates infeasible values early, making the solution process more efficient.
- Instead of blindly searching for a solution, **constraint propagation** actively **reduces the domains** of variables by propagating constraints throughout the problem.



Constraint Propagation: Inference in CSPs

- **Constraint propagation** reduces the number of legal values for a variable
 - which in turn can reduce the legal values for another variable, and so on
- This process helps to detect inconsistencies early and narrows down the possible assignments for variables.
- **How?**
 - ***Key idea:*** local consistency



Constraint Propagation: Inference in CSPs

- **Constraint propagation**
 - iteratively removes inconsistent values from variable domains based on constraints.
- **Constraint propagation works using 3 types of *local consistency*:**
 - ***Node Consistency*** (Unary Constraints)
 - ***Arc Consistency*** (Binary Constraints)
 - ***Path Consistency*** (Higher-order Constraints)



Node Consistency

- A single variable is **node-consistent** if all the values in its domain satisfy its unary constraints.
- Unary constraints restrict values for a single variable.
- Example: Consider our the map coloring problem
 - Constraint: *South Australians dislike green* (i.e. $SA \neq green$)
 - Domain of $SA = \{red, green, blue\}$
 - Domain of SA after applying **node consistency** = $\{red, blue\}$
- A graph is node-consistent if every variable in the graph is node-consistent



Node Consistency

Let us consider a variable, its domain and a constraints as below:

- A Variable X has a domain $\{1,2,3,4\}$ and
- A given node constraint is $X \neq 4$.
- After applying *node consistency*,
 - the new domain of X is $\{1,2,3\}$.

It is easy to eliminate all the unary constraints in a CSP by reducing the domain of variables with unary constraints at the start of the solving process

Since X cannot have value 4, 4 is removed from the domain.



Arc Consistency

- A variable in a CSP is **arc-consistent** if every value in its domain satisfies the variable's binary constraints
- A variable X_i is arc-consistent w.r.t. another variable X_j if for every value in the current domain D_i there is some value in the domain D_j that satisfies the binary constraint on the arc (X_i, X_j) .
- A graph is arc-consistent if every variable is arc-consistent with every other variable.
- The most popular algorithm for enforcing arc consistency is called **AC-3**



Arc Consistency

Let us consider two variables X and Y , their domains and a binary constrain as below:

- Domain of X : $\{1,2,3\}$ and Domain of Y : $\{1,3\}$
- Binary constraint: $X > Y$
 - **Arc-consistency:** Since only for $X = 2, 3$ and $Y = 1$, the constrains $X > Y$ becomes true, 1 is removed from X and 3 is removed from Y .
- After applying *arc consistency*:
 - Domain of X : $\{2,3\}$ and Domain of Y : $\{1\}$



AC-3

- The **AC-3 algorithm** (Arc Consistency 3) is a **constraint propagation algorithm**
- It is used in **Constraint Satisfaction Problems (CSPs)** to make the domains of variables arc-consistent.



AC-3

Algorithm 1 AC-3 Algorithm

```

1: procedure AC-3( $CSP$ )
2:    $Q \leftarrow$  all arcs  $(X_i, X_j)$  in the CSP
3:   while  $Q$  is not empty do
4:     Remove an arc  $(X_i, X_j)$  from  $Q$ 
5:     if REVISE( $X_i, X_j$ ) then
6:       if  $Domain[X_i]$  is empty then
7:         return failure
8:       end if
9:       for each neighbor  $X_k$  of  $X_i$  where  $X_k \neq X_j$  do
10:        Add  $(X_k, X_i)$  to  $Q$ 
11:      end for
12:    end if
13:  end while
14:  return success
15: end procedure

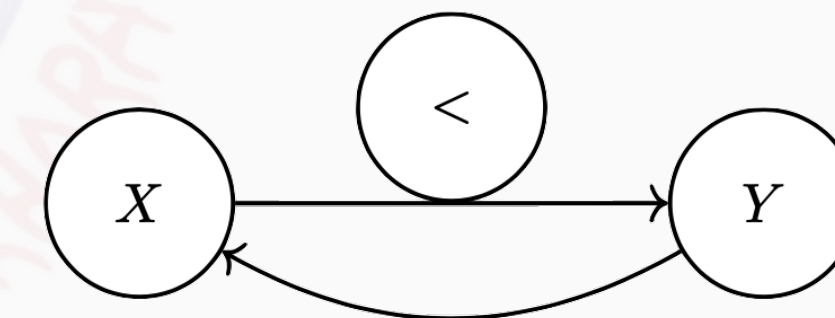
```

```

1: procedure REVISE( $X_i, X_j$ )
2:    $revised \leftarrow false$ 
3:   for each  $a \in Domain[X_i]$  do
4:     if no  $b \in Domain[X_j]$  satisfies constraint  $C_{ij}(a, b)$  then
5:       remove  $a$  from  $Domain[X_i]$ 
6:        $revised \leftarrow true$ 
7:     end if
8:   end for
9:   return  $revised$ 
10: end procedure

```

Constraint Graph Example: $X < Y$



Domains:

$$D(X) = \{1, 2, 3\}, \quad D(Y) = \{2, 3\}$$

Constraint: $X < Y$

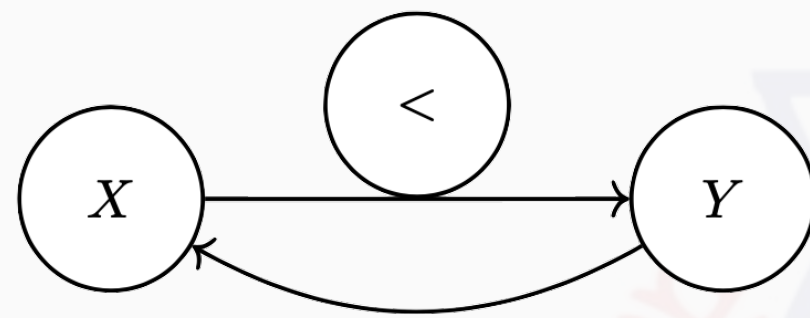
After AC-3:

$$D(X) = \{1, 2\}, \quad D(Y) = \{2, 3\}$$



AC-3

Constraint Graph Example: $X < Y$



Domains:

$$D(X) = \{1, 2, 3\}, \quad D(Y) = \{2, 3\}$$

Constraint: $X < Y$

After AC-3:

$$D(X) = \{1, 2\}, \quad D(Y) = \{2, 3\}$$

CSP:

- Variables: X, Y
- Domains:
 - $D(X) = \{1, 2, 3\}$
 - $D(Y) = \{2, 3\}$
- Constraint: $X < Y$

Step-by-step:

1. Initial arc queue: $(X, Y), (Y, X)$

2.

Check (X, Y) :

- $1 < 2, 1 < 3 \rightarrow \text{OK}$
- $2 < 2$ **✗**, $2 < 3 \rightarrow \text{OK}$
- $3 < 2, 3 < 3$ **✗** $\rightarrow$ Remove 3 from $D(X)$
- $D(X) = \{1, 2\}$
- Revised = true $\rightarrow$ Add neighbors of X (Y) back to queue $\rightarrow$ Add (Y, X)

3.

Check (Y, X) :

- For each b in $D(Y) = \{2, 3\}$, is there an a in $D(X) = \{1, 2\}$ such that $a < b$?
- 2: $1 < 2 \rightarrow \text{OK}$
- 3: $1 < 3, 2 < 3 \rightarrow \text{OK} \rightarrow$ No change

4. Queue is empty $\rightarrow$ Domains are arc-consistent.



Path Consistency

- In our Map coloring Example:
 - Western Australia, Northern Territory, and South Australia all touch each other
 - So we need at least three colors for them alone
 - $\therefore$ node-consistency or arc-consistency does not work
- ***Path consistency*** solve this problem, :
 - It tightens the binary constraints by using implicit constraints that are inferred by looking at triples of variables.



Path Consistency

- A two variable set $\{X_i, X_j\}$ is path-consistent wrt a third variable X_m if for every assignment $\{X_i = \alpha, X_j = \beta\}$ consistent with the constraint (if any) on $\{X_i, X_j\}$, there is an assignment X_m that satisfies the constraints on $\{X_i, X_m\}$ and $\{X_m, X_j\}$.
- The name refers to the overall consistency of the path from X_i to X_j with X_m in the middle.



Path Consistency

- Extends arc consistency to triplets of variables.
- Ensures that if X_i and X_j satisfy a binary constraint, there must be a compatible value in X_m
- Let us consider three variables: A , B , C with the following domains and constraints:
 - Domains: $A = \{ 1 , 2 \}$, $B = \{ 1 , 2 \}$, $C = \{ 1 , 2 , 3 \}$
 - Constraints: $A + B = C$ and $B < C$
 - After applying path consistency,
 - the new domain of C : $\{ 2 , 3 \}$



Path Consistency

Let's see how path consistency fares in coloring the Australia map with two colors. We will make the set $\{WA, SA\}$ path-consistent with respect to NT . We start by enumerating the consistent assignments to the set. In this case, there are only two: $\{WA = red, SA = blue\}$ and $\{WA = blue, SA = red\}$. We can see that with both of these assignments NT can be neither *red* nor *blue* (because it would conflict with either WA or SA). Because there is no valid choice for NT , we eliminate both assignments, and we end up with no valid assignments for $\{WA, SA\}$. Therefore, we know that there can be no solution to this problem.



Constraint Propagation- Benefits

- ***Reduces Search Space:***
 - By eliminating inconsistent values early, constraint propagation reduces the number of assignments that need to be explored.
- ***Detects Inconsistencies Early:***
 - If a domain becomes empty during propagation, it indicates that no solution exists, allowing the algorithm to backtrack immediately.
- ***Improves Efficiency:***
 - Constraint propagation can significantly speed up the solving process by narrowing down the possibilities before starting the search.



Backtracking Search for CSPs

- Constraint propagation (like AC-3)
 - significantly reduce the search space by pruning invalid values early (Removes values from domains that are inconsistent with constraints)
 - **Does not choose** variable assignments
 - if **multiple values are still valid** after applying constraint propagation, we need to **search** through them to find the right one
 - Example: After AC-3, $D(X) = \{1,2\}$ and $D(X) = \{2,3\}$
- There are still **valid choices**, and we must try them to find which combination satisfies all constraints. **Backtracking** is the systematic way to explore them.



Backtracking Search for CSPs

- Best Practice to Solve CSPs

Constraint Propagation + Backtracking Search + Heuristics

- Before/during search, use:
 - AC-3 or Forward Checking
 - Variable Ordering Heuristics (MRV, Degree)
 - Value Ordering Heuristics (Least Constraining Value)
- This combination dramatically reduces the number of nodes explored and increases the efficiency of solving



Backtracking Search for CSPs

- Systematically explores possible variable assignments to find a complete assignment that satisfies **all constraints**.
- chooses values for one variable at a time and backtracks when a variable has no legal values left to assign
- **Goal:** To find a complete and consistent assignment of values to variables such that **every constraint is satisfied**.



Backtracking Search for CSPs

Algorithm: BACKTRACKING SEARCH

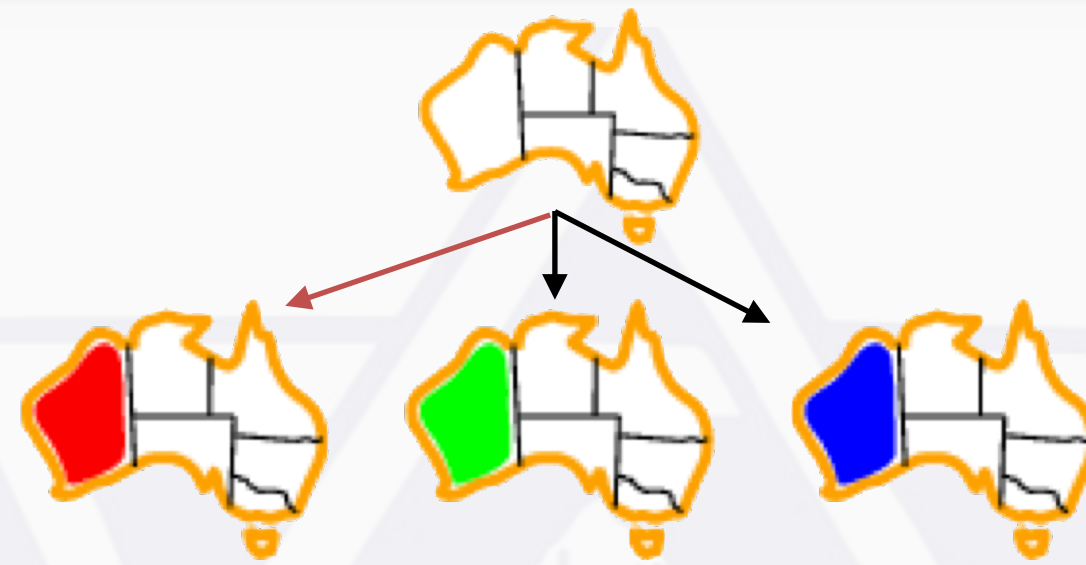
1. Select an Unassigned Variable:
 - i. Choose a variable that has not yet been assigned a value.
2. Assign a Value:
 - ii. Assign a value from the variable's domain.
3. Check Constraints:
 - iii. If the assignment satisfies all constraints, move to the next variable.
 - iv. If the assignment violates a constraint, backtrack and try a different value.
4. Backtrack:
 - v. If no valid value is found for the current variable, backtrack to the previous variable and try a different value.
5. Repeat this process until a complete assignment is found or all possibilities are exhausted.



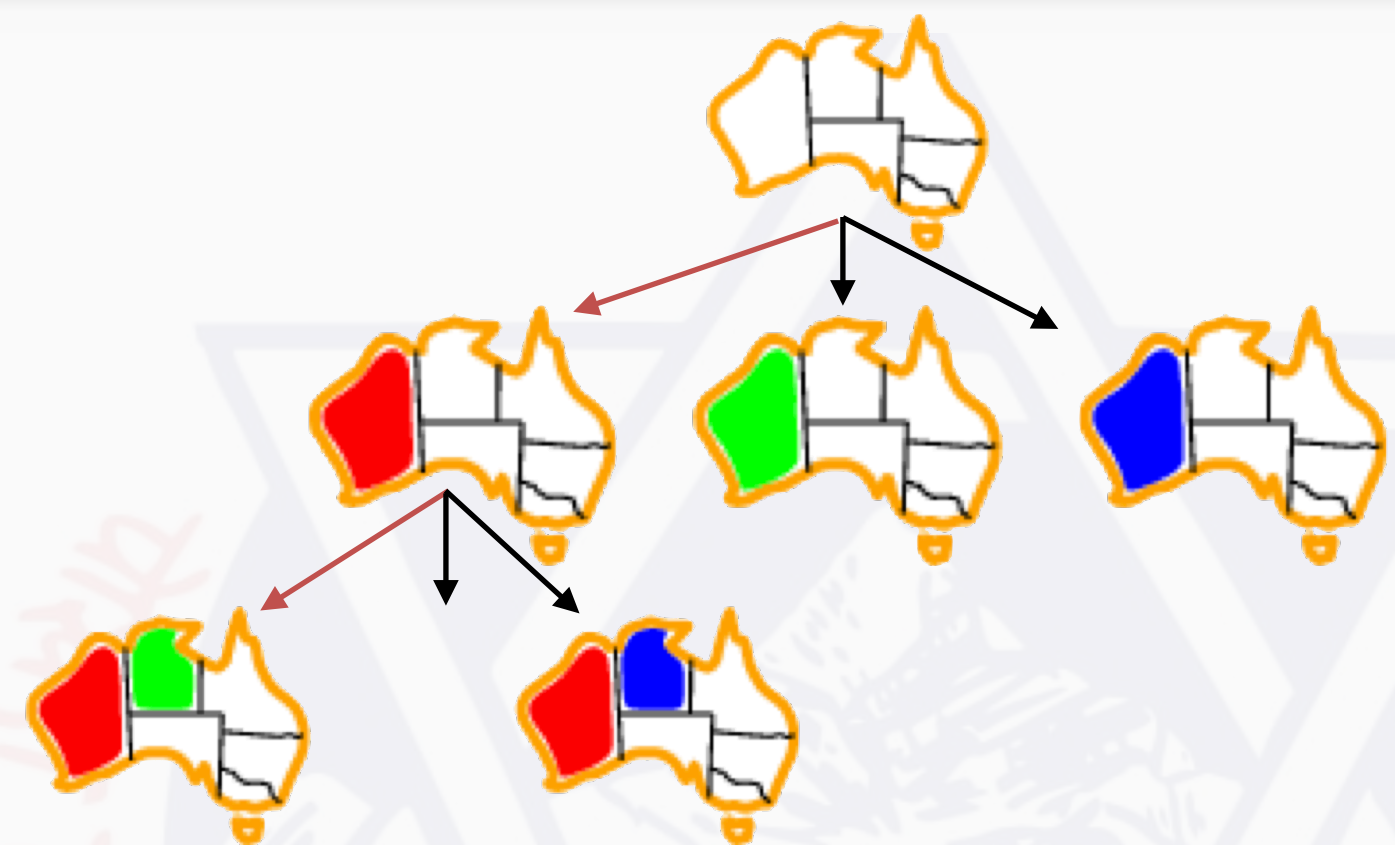
Example: Map Coloring Problem

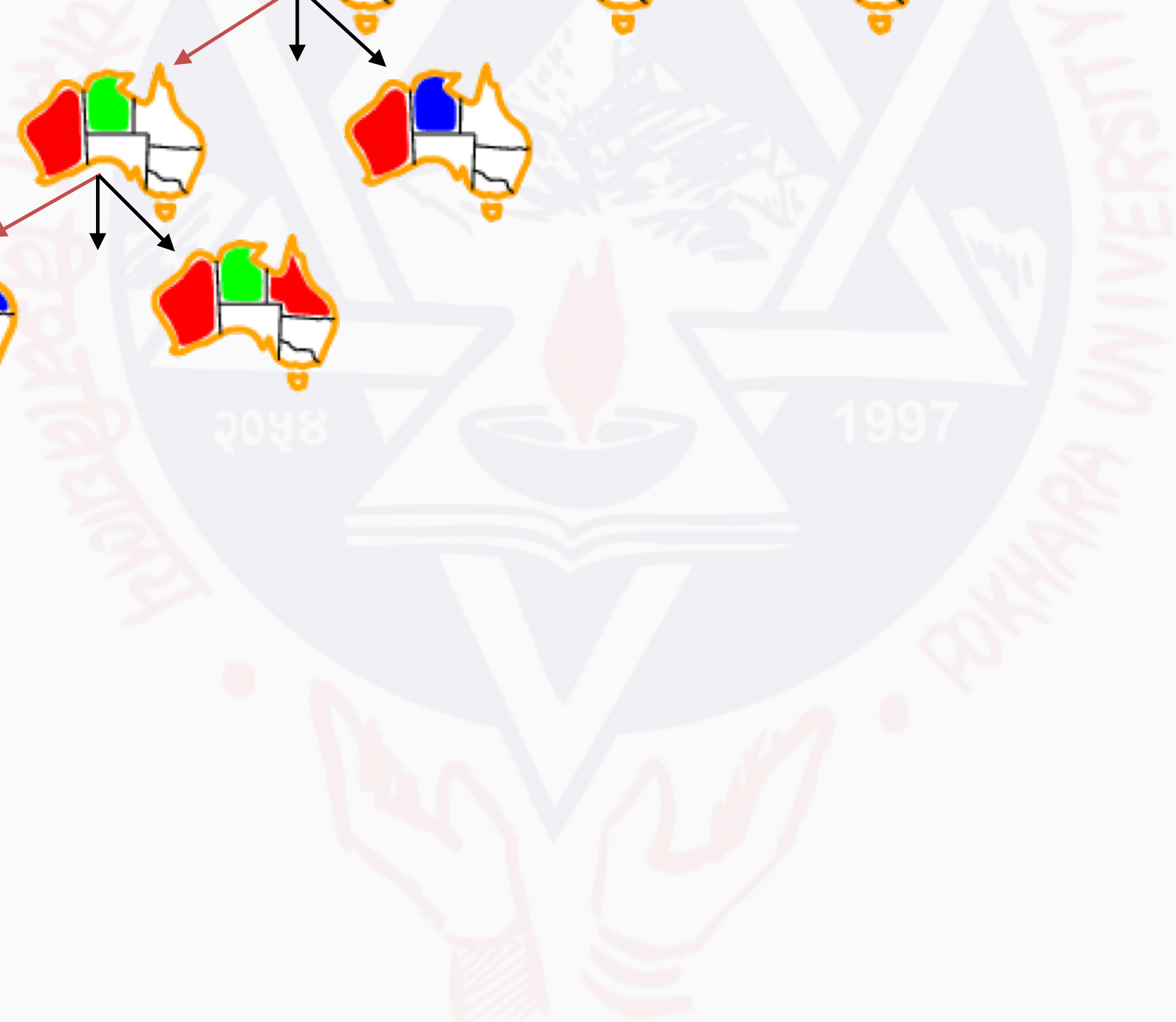


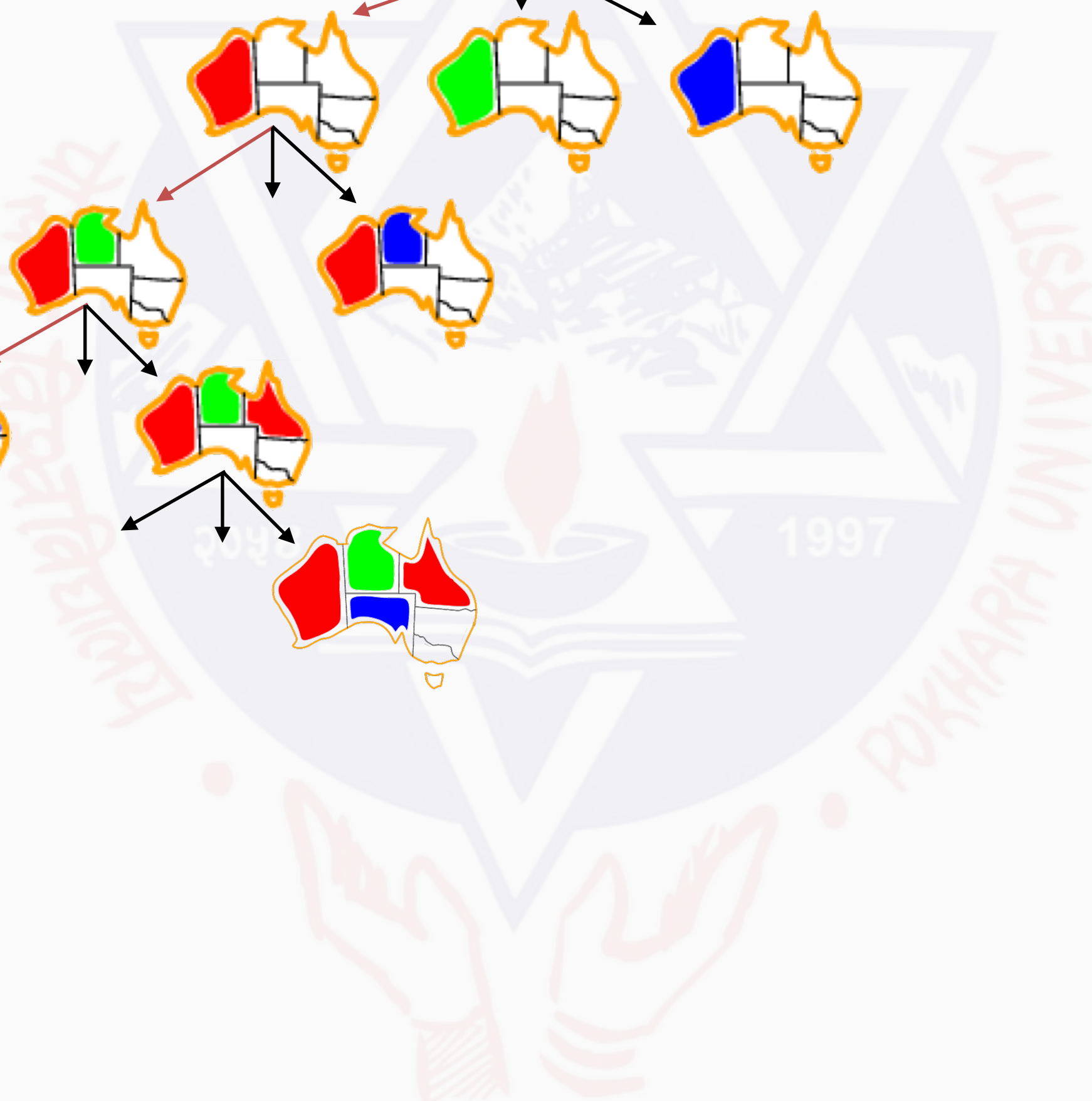
Example: Map Coloring Problem

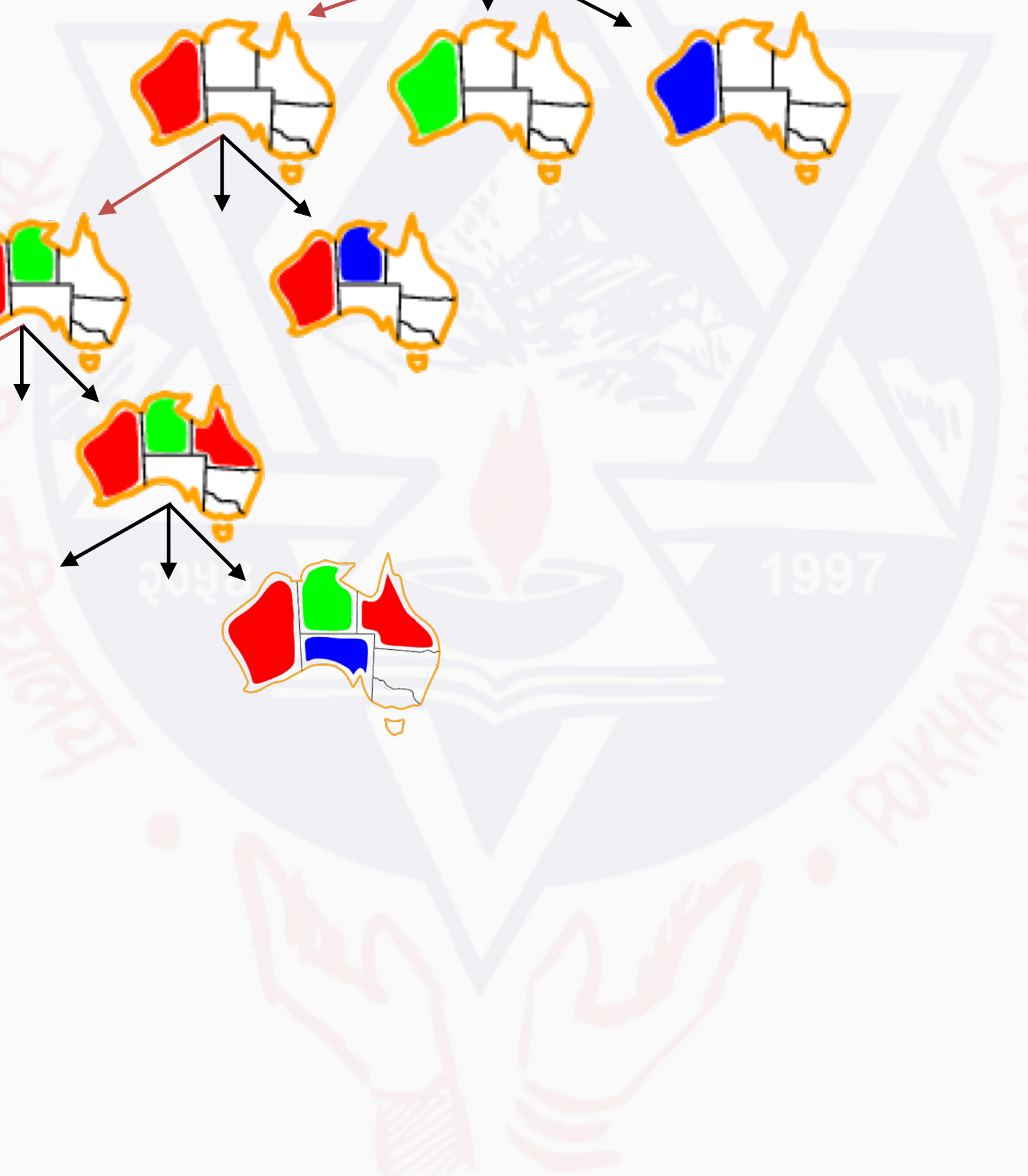


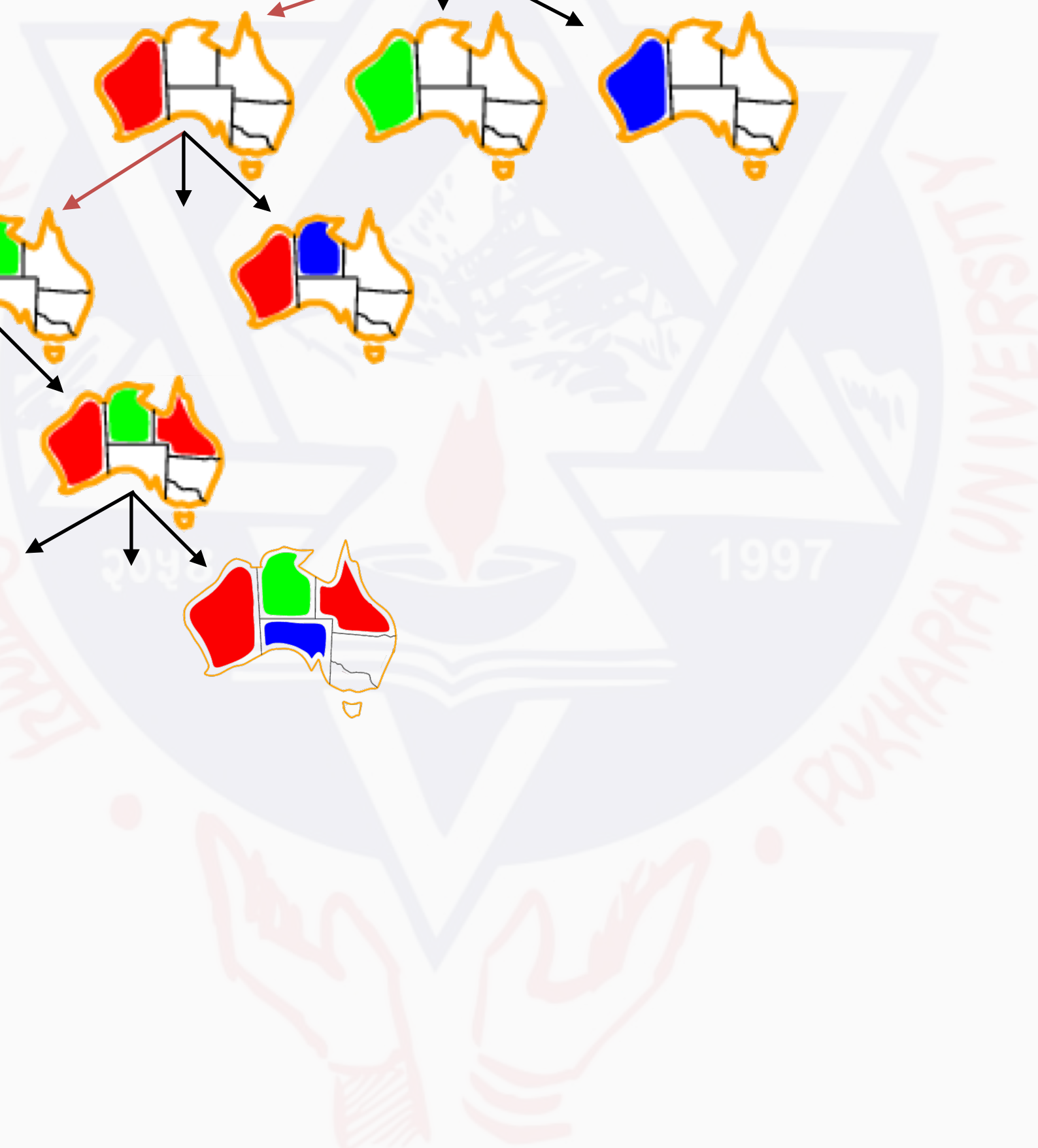
Example: Map Coloring Problem

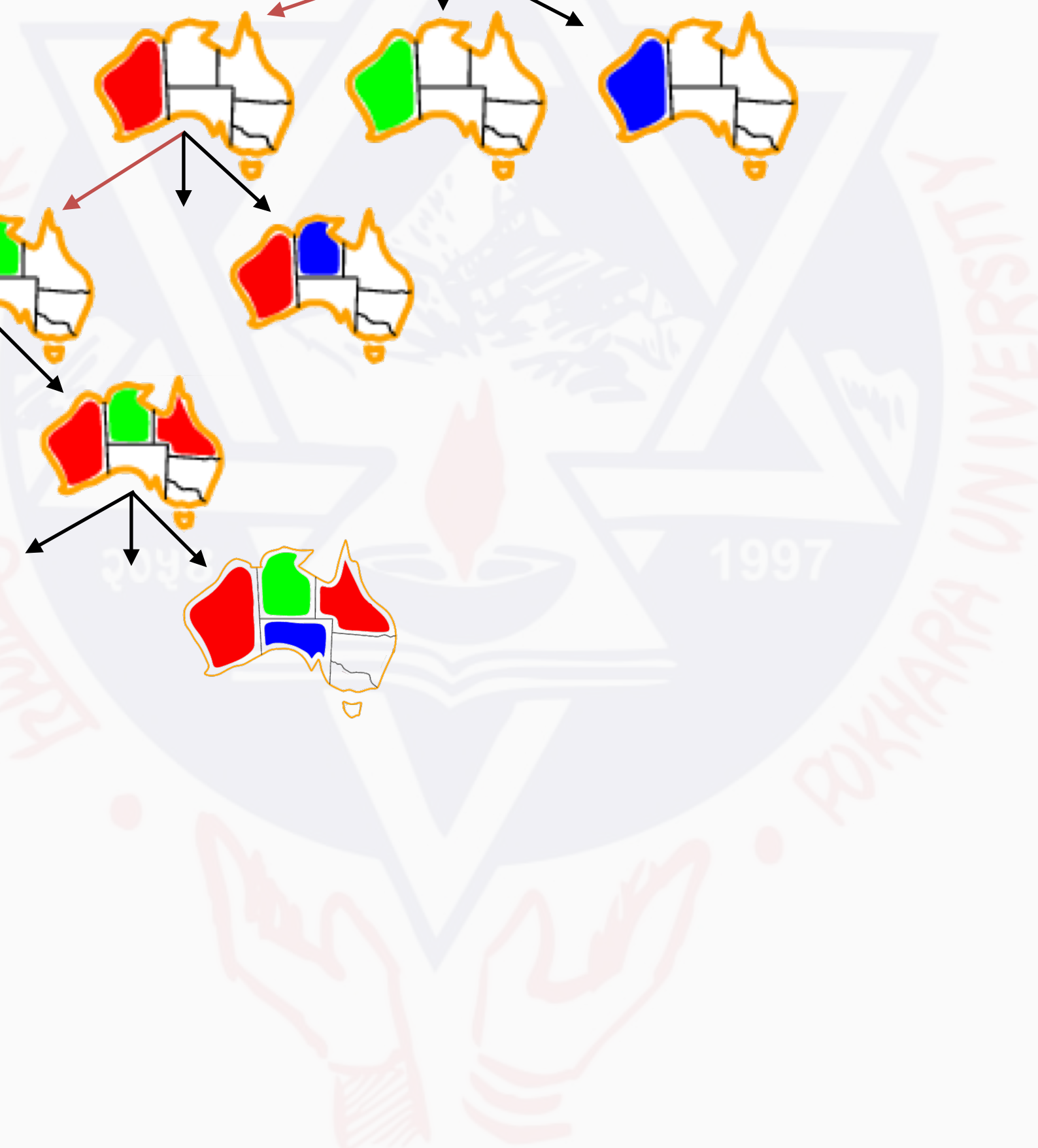








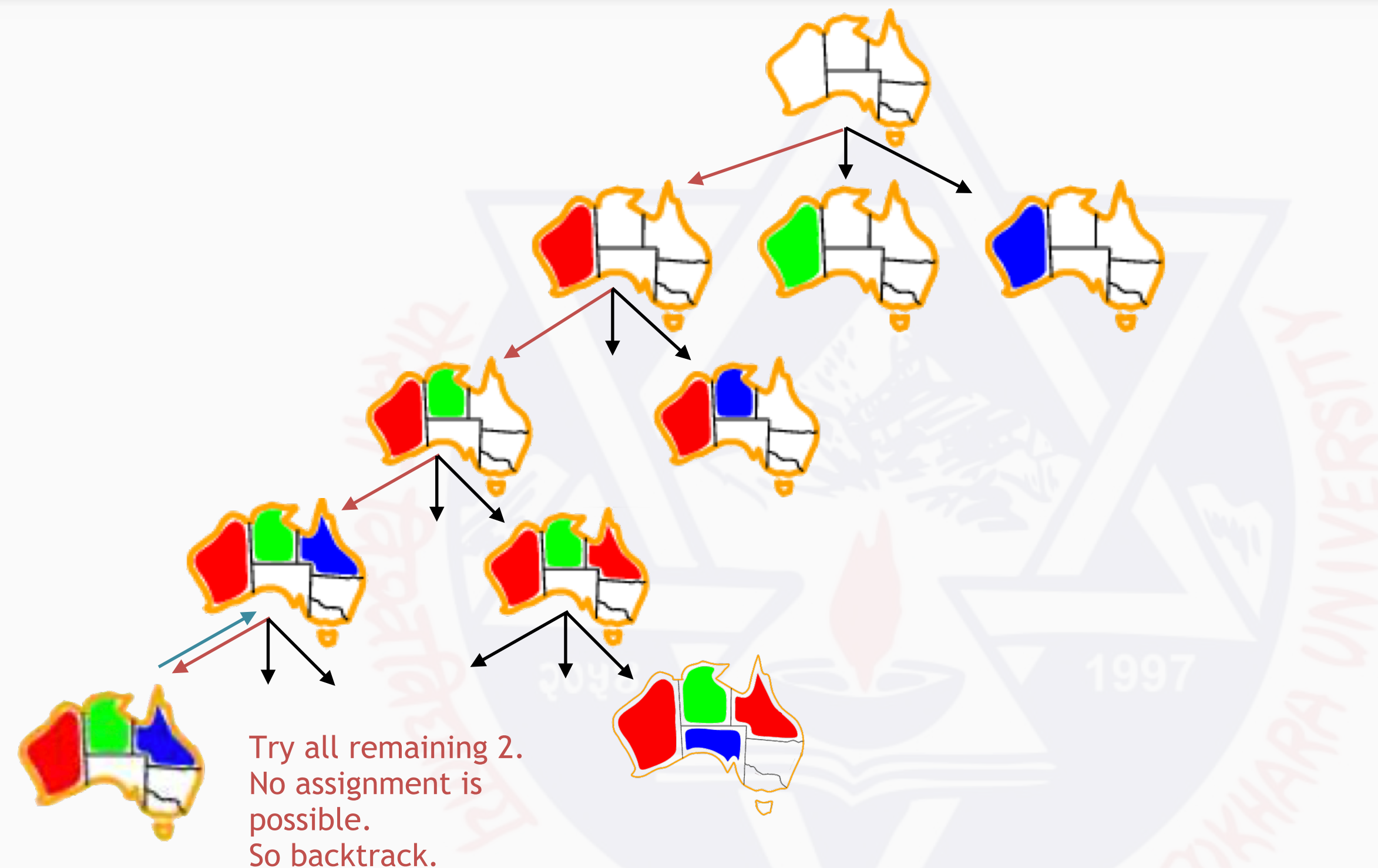


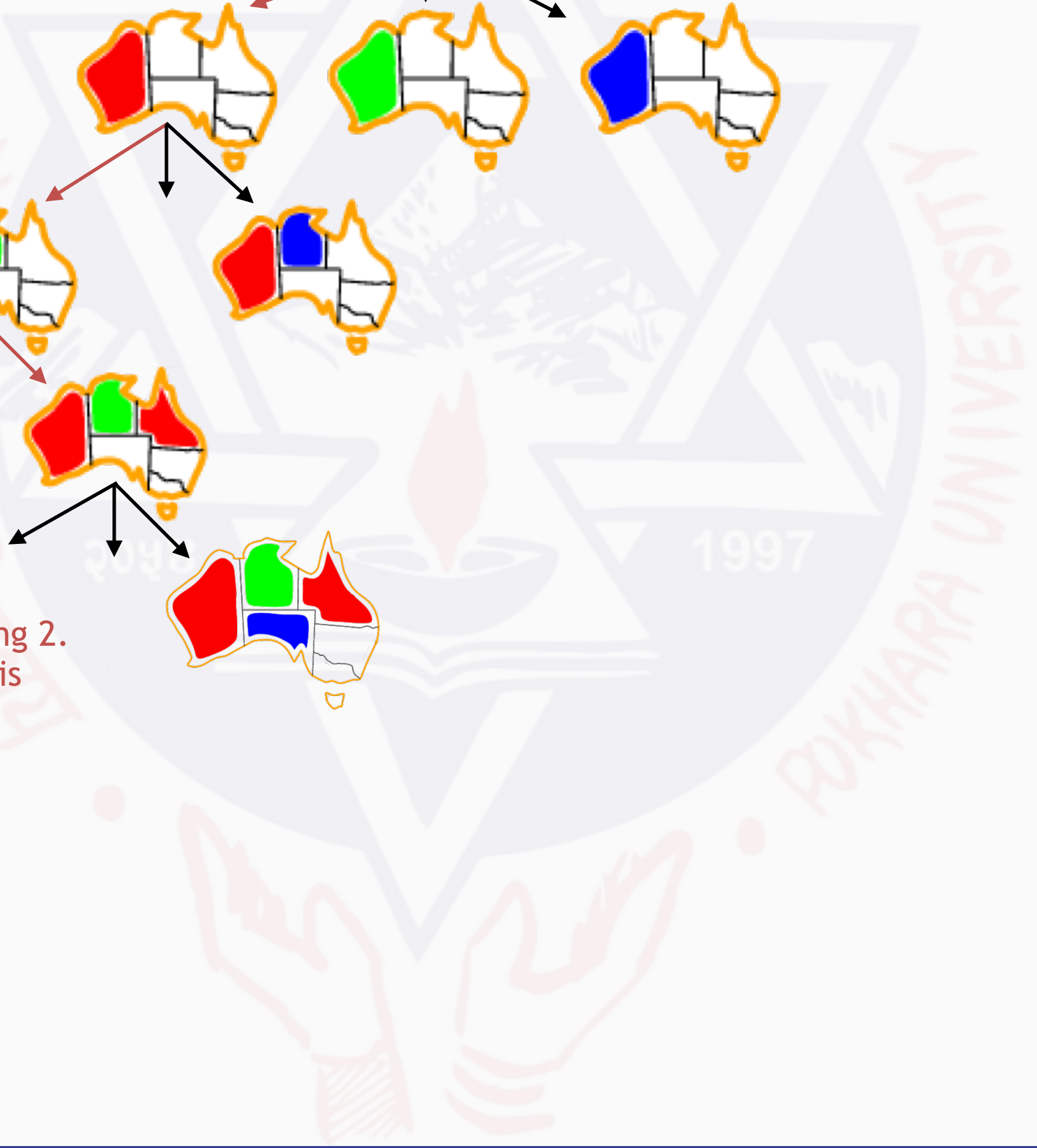


No assignment is possible
For SA. So Backtrack.

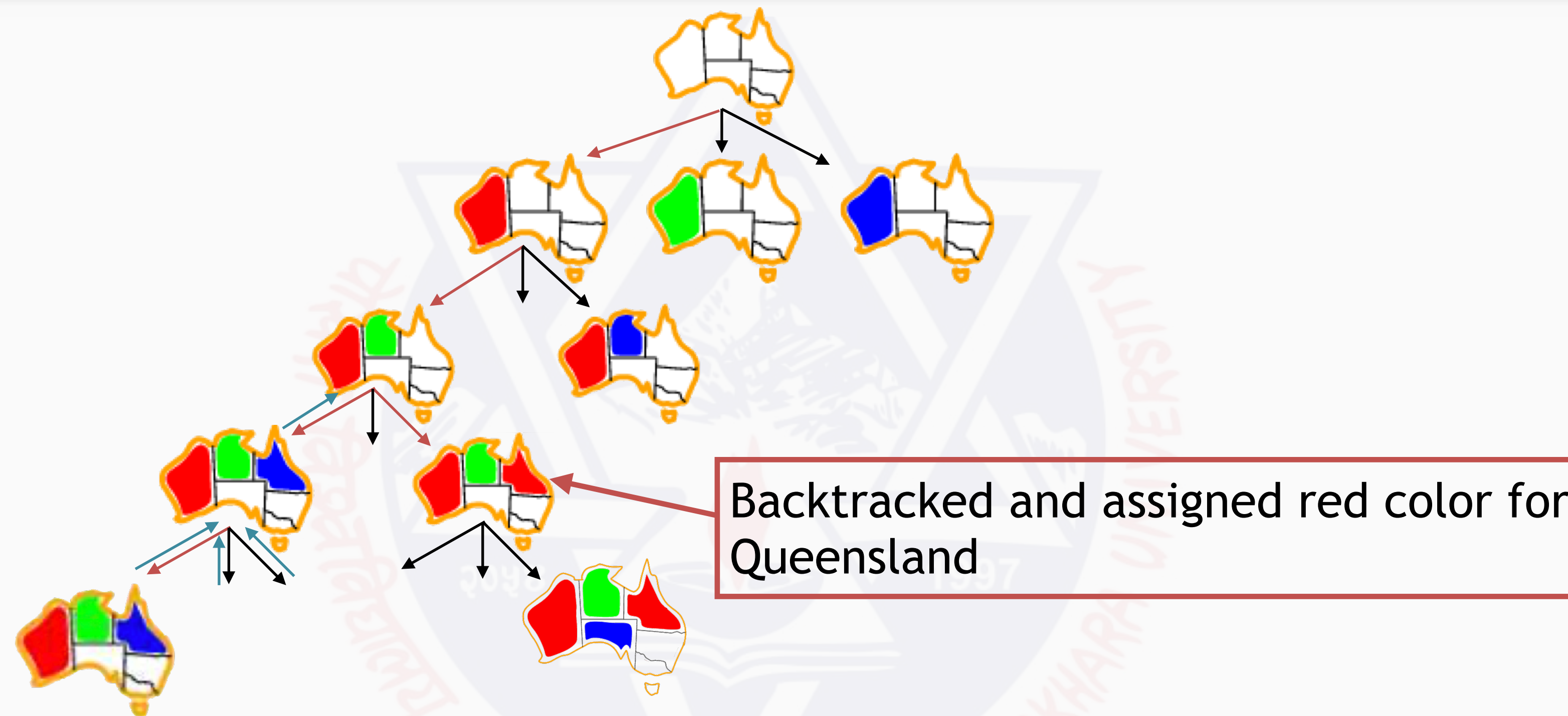


Example: Map Coloring Problem

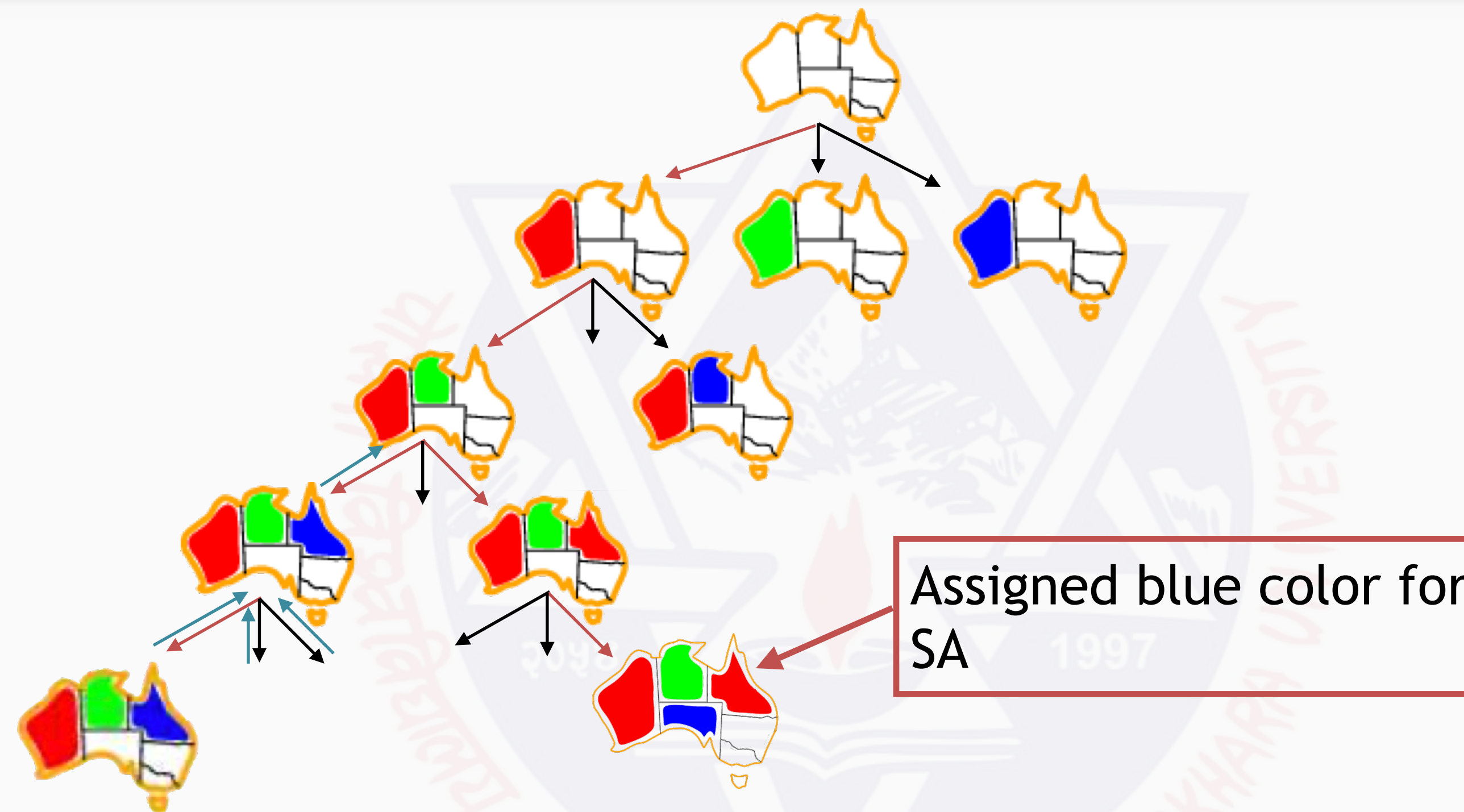


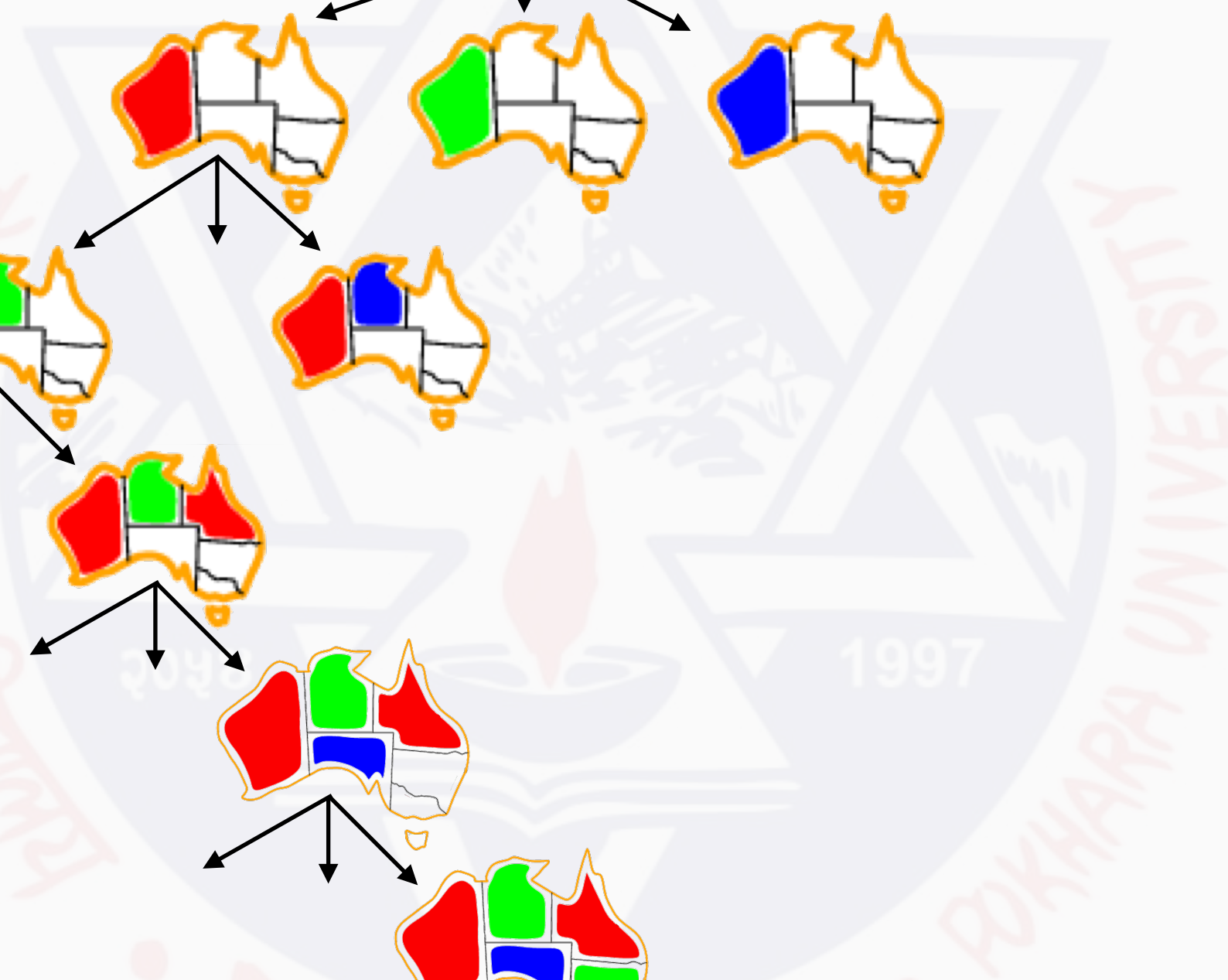


Example: Map Coloring Problem



Example: Map Coloring Problem





Improving Backtracking Efficiency

- **General-purpose** methods can give huge gains in speed:
 - Which variable should be assigned next?
 - In what order should its values be tried?
 - Can we detect inevitable failure early?



Most Constrained Variable

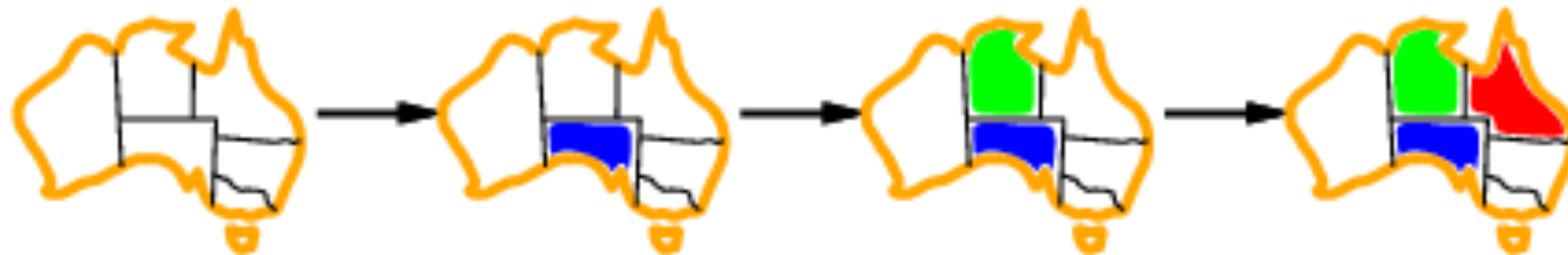
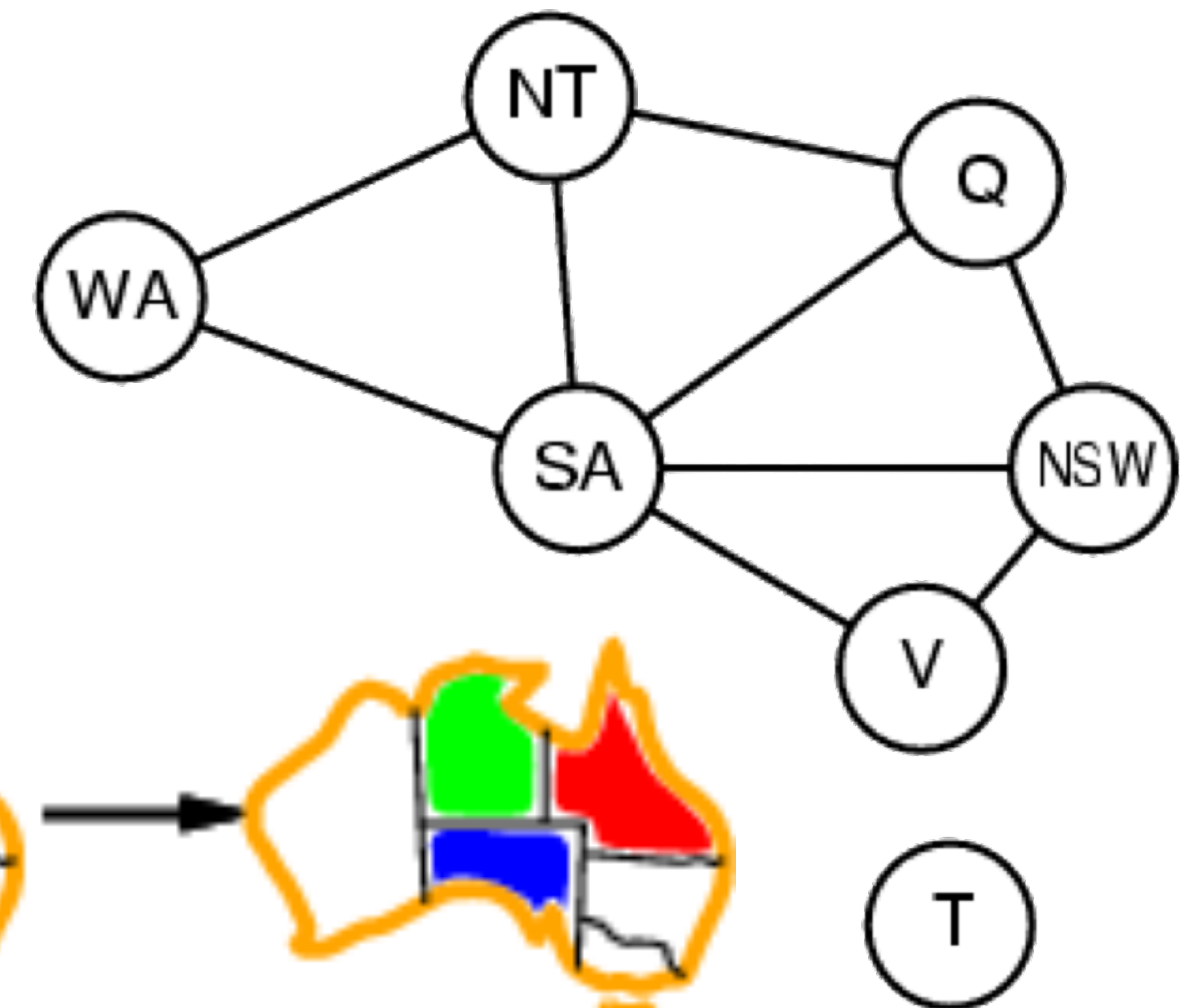
- Most constrained variable:
 - choose the variable with the fewest legal values



- Also known as **minimum remaining values (MRV)** or **fail-first** heuristic
- Picks a variable which will cause failure as soon as possible, allowing the tree to be pruned.

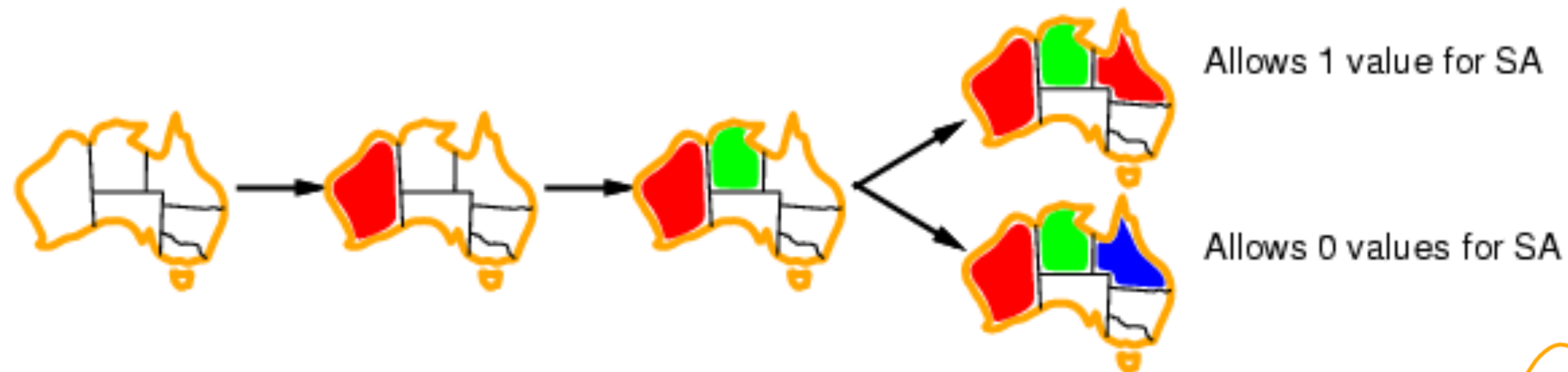
Most Constraining Variable

- Most constraining variable:
 - choose the variable with the most constraints on remaining variables (most edges in graph i.e. SA)
 - also called degree heuristics
- Tie-breaker among most constrained variables



Least Constraining Value

- Given a variable, choose the least constraining value:
 - the one that rules out the fewest values in the remaining variables



- Leaves maximal flexibility for a solution.
- Combining these heuristics makes 1000 queens feasible

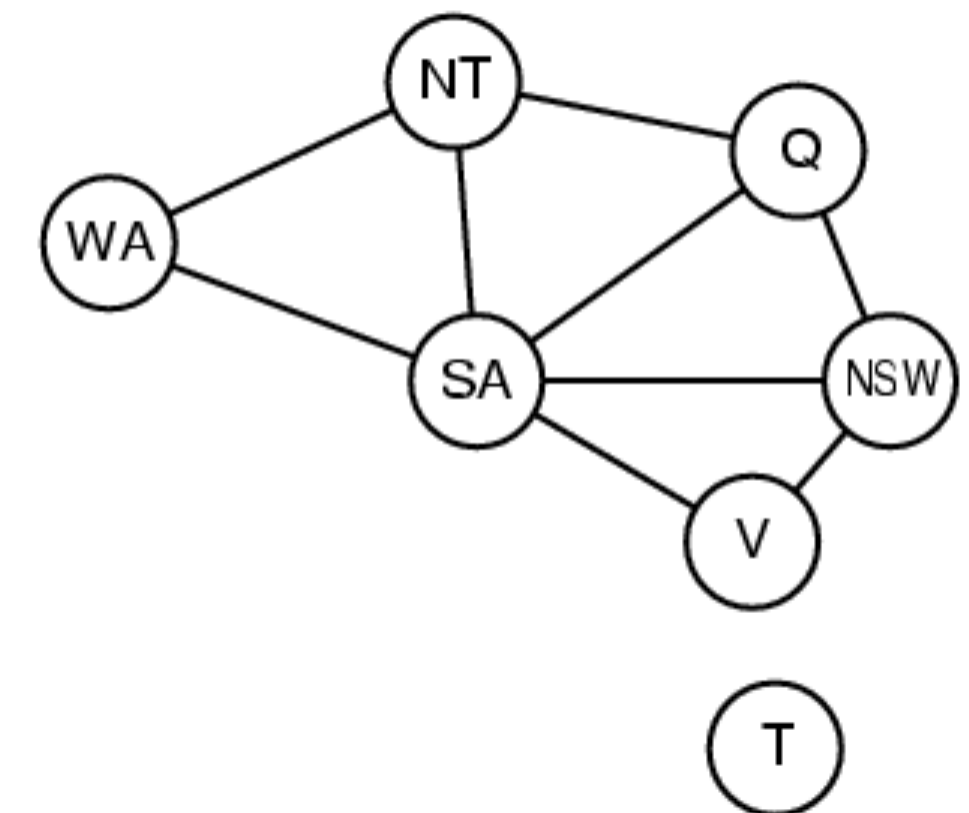


Forward Checking

(Propagating Information through Constraints)

Idea:

- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values

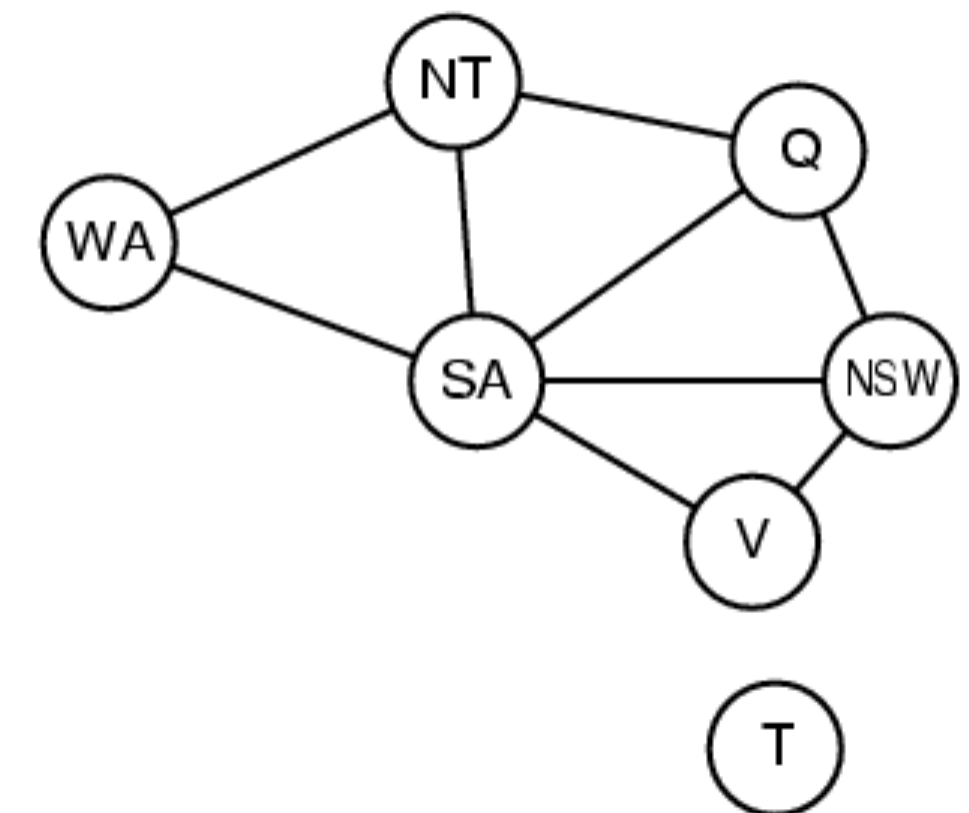


Forward Checking

(Propagating Information through Constraints)

Idea:

- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values



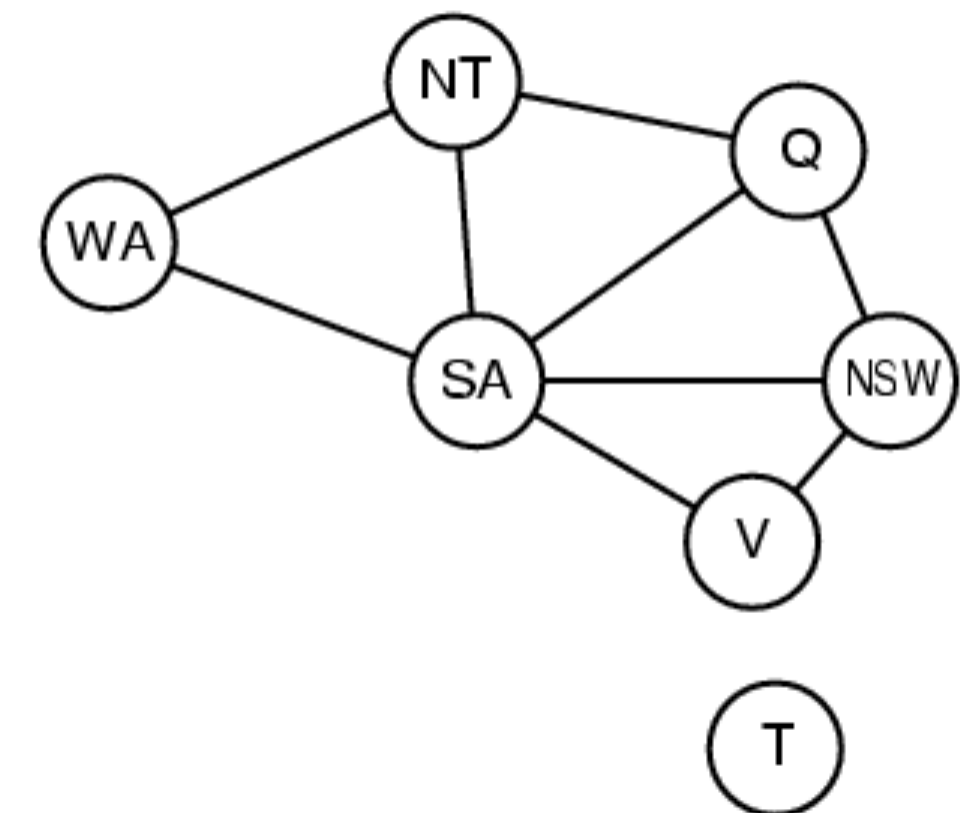
WA	NT	Q	NSW	V	SA	T

Forward Checking

(Propagating Information through Constraints)

Idea:

- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values



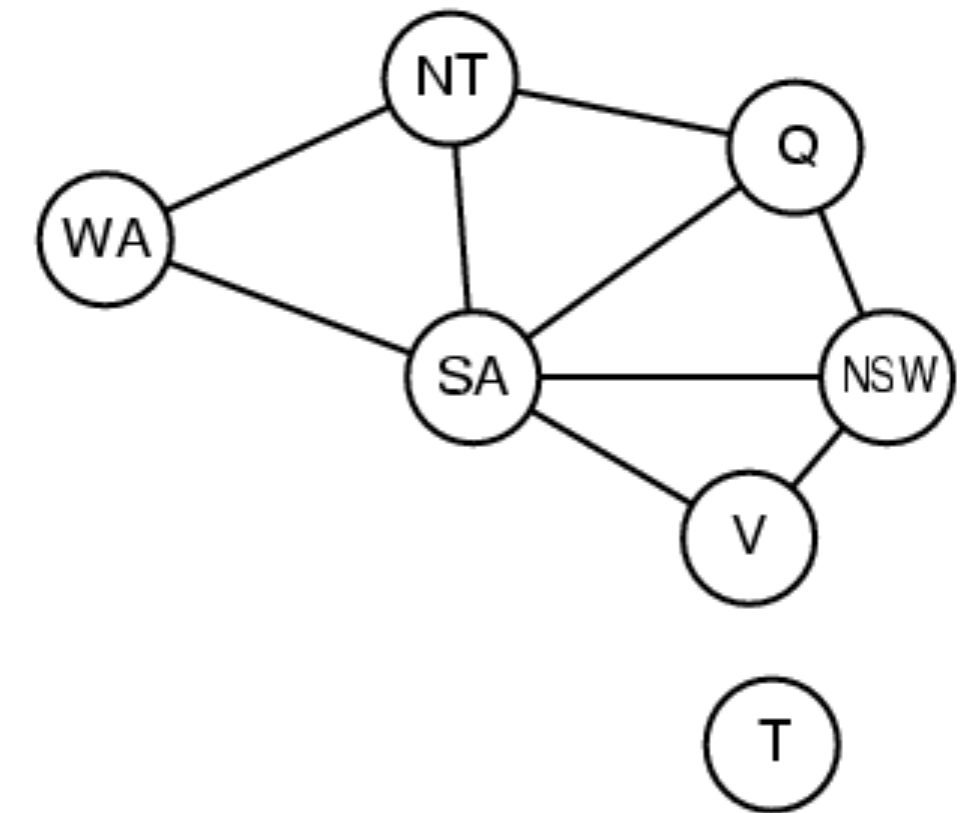
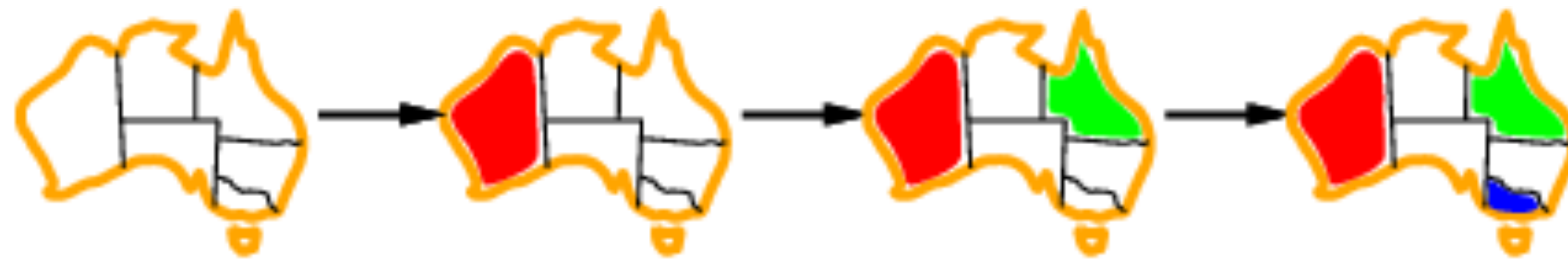
WA	NT	Q	NSW	V	SA	T

Forward Checking

(Propagating Information through Constraints)

Idea:

- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values



WA	NT	Q	NSW	V	SA	T

Forward Checking

- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:

Constrain Propagation / Backtracking Search

Technique	Purpose	What it does	Limitation
Constraint Propagation (e.g., AC-3)	Reduce the search space	Removes values from domains that are inconsistent with constraints	Does not choose variable assignments
Backtracking Search	Find a complete solution	Systematically tries combinations of variable assignments	Inefficient without propagation





THANK YOU